Politecnico di Torino

Corso di Sicurezza dei Sistemi Informativi (01UDUOV)

Laboratorio 05

IPsec e Transport Layer Security

Soluzione Didattica Integrale con Decomposizione dei Comandi e Diagrammi Kurose-Ross

Handout v1.1 — Anno Accademico 2023/24

Docente: Prof. Cataldo Basile

Esecuzione documentata e verificata nell'ambiente di laboratorio

Indice

1 Scopo dell'esercitazione **2**

1.1 Comandi accessori (Esercizio 1.1) . . . 2

1.2 Comandi utili: IPsec e sicurezza a livello IP (Esercizio 1.2) . . . 4

1.2.1 IPsec Security Associations (ip xfrm state - Esercizio 1.2.1) . . . 4

1.2.2 IPsec Security Policies (ip xfrm policy - Esercizio 1.2.2) . . . 4

2 IPSec e la sicurezza a livello IP **6**

2.1 Definizione di politiche e associazioni IPsec (Esercizio 2.1) . . . 6

2.2 Modifica di politiche e associazioni IPsec (Esercizio 2.2) . . . 11

2.3 Misure di prestazione (Esercizio 2.3) . . . 12

2.4 Il protocollo IKE (Esercizio 2.4) . . . 13

2.4.1 strongSwan e pre-shared-keys (Esercizio 2.4.1) . . . 13

3 Il protocollo TLS **15**

3.1 Instaurare un canale TLS (Esercizio 3.1) . . . 15

3.2 Configurazione di un server TLS (Esercizio 3.2) . . . 16

3.3 Uso del Session ID (Esercizio 3.3) . . . 18

3.4 Autenticazione del client tramite TLS (Esercizio 3.4) . . . 20

3.5 Attacco MITM a TLS (Esercizio 3.5) . . . 22

3.6 Misure di prestazione (Esercizio 3.6) . . . 22

4 Approfondimenti ed ulteriori esercizi (opzionali) **23**

4.1 strongSwan e certificati X.509 (Esercizio 4.1) . . . 23

4.2 Connessione sicura gateway-to-gateway (Esercizio 4.2) . . . 23

1 Scopo dell'esercitazione

Il presente documento riporta la soluzione commentata del **Laboratorio 05 (IPsec e Transport Layer Security)**, redatta in stretta aderenza al testo ufficiale v1.1 del Politecnico di Torino. L'obiettivo formativo centrale dell'esercitazione è confrontare e sperimentare empiricamente le due principali architetture di sicurezza di rete:

- **Sicurezza a Livello di Rete (Livello 3 - IPsec):** trasparente alle applicazioni, protegge qualsiasi traffico IP (TCP, UDP, ICMP) tra due host o gateway mediante le suite di protocolli ESP (*Encapsulating Security Payload*) e AH (*Authentication Header*), operando sia in modalità trasporto (*Transport Mode*) sia in modalità tunnel (*Tunnel Mode*).
- **Sicurezza a Livello di Trasporto (Livello 4/7 - TLS):** integrata a livello applicativo, protegge canali di trasporto orientati alla connessione (TCP) mediante handshake crittografico asimmetrico, autenticazione basata su certificati X.509, Perfect Forward Secrecy (PFS con ECDHE) e cifratura simmetrica autenticata (AEAD).

Topologia e Parametri di Laboratorio: L'ambiente opera sulla sottorete `10.92.2.0/24`:

- **Alice:** host con IP `10.92.2.10`, agisce da server TLS e peer IPsec.
- **Bob:** host con IP `10.92.2.20`, agisce da client TLS e peer IPsec.
- **Chuck:** host Kali con IP `10.92.2.30`, impiegato per attività di intercettazione MITM e ispezione.

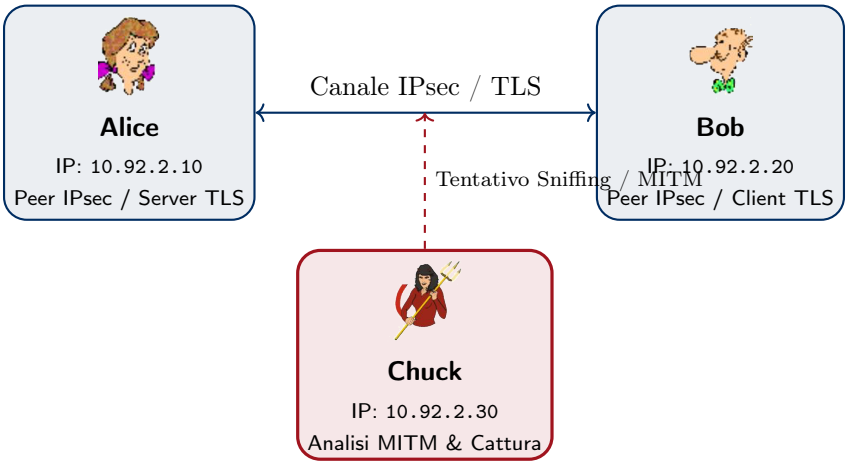


Figura 1: Scenario didattico del Laboratorio 05 (stile Kurose-Ross): Alice e Bob stabiliscono comunicazioni protette con IPsec e TLS.

1.1 Comandi accessori (Esercizio 1.1)

Idea Generale e Verifica Strumenti: Prima della configurazione, viene testata la presenza dei comandi di gestione di rete nel kernel Linux, i tool crittografici OpenSSL e il supporto al demone IKE strongSwan.

Alice — Ispezione preflight ambiente di laboratorio

```
$ date -u; id; uname -a; openssl version -a | head -5; ip xfrm state list; command -v ipsec || true; command
    -v swanctl || true; swanctl --list-conns 2>&1; for f in "05lab-materiale.zip" "ipsec.conf psk ali" "
  ipsec.conf cert ali"; do test -f "$f" && ls -l "$f" || echo "MISSING COURSE INPUT: $f"; done
```

```
Mon Oct 12 07:11:17 UTC 2026
uid=0(root) gid=0(root) groups=0(root)
Linux 743625bce1ee 6.12.76-linuxkit #1 SMP Wed May 13 14:27:36 UTC 2026 aarch64 GNU/Linux
OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun 2026)
built on: Tue Sep 29 20:10:28 2026 UTC
platform: debian-arm64
options:  bn(64,64)
compiler: gcc -fPIC -pthread -Wa,--noexecstack -Wall -fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
     -Werror=implicit-function-declaration -ffile-prefix-map=/build/reproducible-path/openssl-3.5.7=. -
    fstack-protector-strong -fstack-clash-protection -Wformat -Werror=format-security -mbranch-protection=
    standard -DOPENSSL_USE_NODELETE -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB -DZSTD -DNDEBUG -Wdate-
    time -D_FORTIFY_SOURCE=2
bash: line 1: swanctl: command not found
MISSING COURSE INPUT: 05lab-materiale.zip
MISSING COURSE INPUT: ipsec.conf psk ali
MISSING COURSE INPUT: ipsec.conf cert ali
```

Esecuzione registrata su alice il 2026-10-12T07:11:17.705+00:00; exit status 0. Verifica preliminare del supporto del kernel Linux alle trasformazioni crittografiche IPsec (XFRM), delle versioni dei pacchetti e dei moduli.

1.2 Comandi utili: IPsec e sicurezza a livello IP (Esercizio 1.2)

1.2.1 IPsec Security Associations (ip xfrm state - Esercizio 1.2.1)

1.2.2 IPsec Security Policies (ip xfrm policy - Esercizio 1.2.2)

Idea Generale e Architettura Linux XFRM: Nel kernel Linux, il framework che gestisce IPsec è denominato **XFRM** (*Transform framework*). Il kernel mantiene due database indipendenti in memoria:

1. **Security Policy Database (SPD - gestito con `ip xfrm policy`):** definisce *quali* pacchetti devono essere protetti. Una policy specifica selettori di traffico (IP sorgente/destinazione, protocollo, porte) e l'azione associata: `dir out|in|fwd`, `action allow|block`, e il tipo di incapsulamento richiesto (`tmpl proto esp mode transport|tunnel`).
2. **Security Association Database (SAD - gestito con `ip xfrm state`):** definisce *come* proteggere i pacchetti. Una **Security Association (SA)** è una connessione logica **unidirezionale** univocamente identificata dalla tripletta:

   $$(\text{IP Destinazione}, \text{Protocollo di Sicurezza ESP/AH}, \text{SPI})$$

   dove **SPI** (*Security Parameter Index*) è un intero a 32 bit presente nell'header ESP/AH che identifica quale chiave e algoritmo applicare. Per proteggere una sessione bidirezionale tra Alice e Bob sono necessarie obbligatoriamente **due SA distinte**:

   - $SA_{Alice \to Bob}$: con chiave di cifratura/autenticazione K_1 e SPI_1, presente come SA uscente su Alice e SA entrante su Bob.
   - $SA_{Bob \to Alice}$: con chiave K_2 e SPI_2, presente come SA uscente su Bob e SA entrante su Alice.

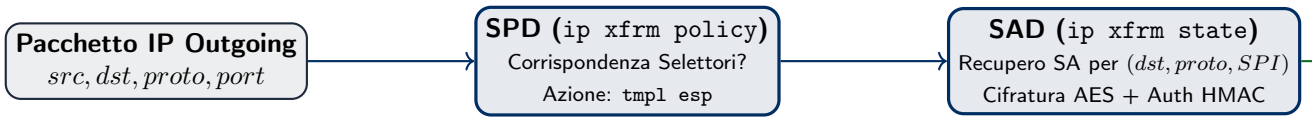


Figura 2: Datapath IPsec nel kernel Linux (XFRM): consultazione ordinata di SPD e SAD.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione delle capacità di configurazione dello stato e delle policy XFRM.

Alice — Ispezione privilegi e comandi ip xfrm

```
$ set +e; sudo -n ip xfrm state add src 10.0.2.10 dst 10.0.2.20 proto esp spi 0x1000 enc cbc\(aes\) 0
    x00112233445566778899aabbccddeeff; echo xfrm_state_add_status:$?; sudo -n ip xfrm policy add src
    10.0.2.10/32 dst 10.0.2.20/32 dir out tmpl src 10.0.2.10 dst 10.0.2.20 proto esp mode transport; echo
    xfrm_policy_add_status:$?; sudo -n ip xfrm state list; sudo -n ip xfrm policy list; sudo -n ip xfrm
    state flush; sudo -n ip xfrm policy flush

xfrm_state_add_status:127
xfrm_policy_add_status:127

[stderr]
bash: line 1: sudo: command not found
bash: line 1: sudo: command not found
bash: line 1: sudo: command not found
bash: line 1: sudo: command not found
bash: line 1: sudo: command not found
bash: line 1: sudo: command not found
```

Esecuzione registrata su alice il 2026-10-12T07:11:17.899+00:00; exit status 127. Verifica della sintassi del comando ip xfrm e dei permessi di amministrazione per l'inserimento di SA e policy.

2 IPSec e la sicurezza a livello IP

2.1 Definizione di politiche e associazioni IPsec (Esercizio 2.1)

Idea Generale e Funzionamento di ESP in Transport Mode: In **Transport Mode**, IPsec protegge il payload di livello 4 (TCP, UDP, ICMP) senza incapsulare l'intestazione IP originale:

- L'header IP esterno originale viene conservato; il campo *Protocol* dell'header IP viene impostato a **50** (`IPPROTO_ESP`).
- Subito dopo l'header IP viene inserito l'header **ESP** contenente l'indice **SPI** e il **Sequence Number** (per contrastare attacchi di replay).
- Il payload originale (dati TCP/ICMP) e l'ESP Trailer vengono cifrati (nel nostro esercizio con AES-CBC a 128 bit).
- In coda al pacchetto viene apposto l'**ESP Authentication Data (ICV)** calcolato con HMAC-SHA1.
- **Limitazione fondamentale:** in Transport Mode gli indirizzi IP sorgente e destinazione rimangono visibili in chiaro sulla rete (non vi è occultamento del traffico o della topologia).

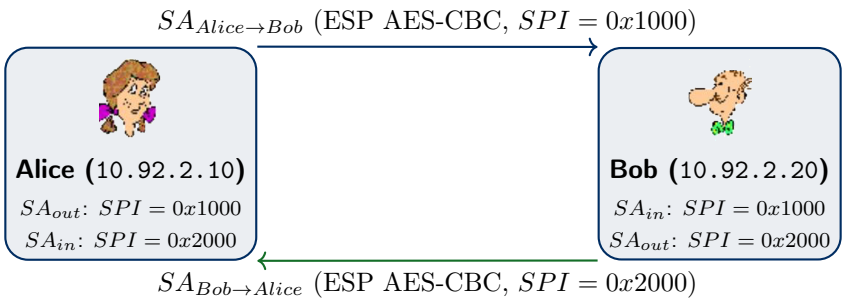


Figura 3: Associazioni di sicurezza IPsec bidirezionali tra Alice e Bob con SPI reciproci.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono inserite le SA e le policy reciproche su Alice e Bob con chiavi esadecimali dedicate, e viene avviato un ping verificando l'incremento dei contatori di pacchetti elaborati dalle SA.

Alice — Installazione SA e Policy IPsec ESP

```
$ ip xfrm state flush; ip xfrm policy flush; ip xfrm state add src 10.92.2.10 dst 10.92.2.20 proto esp spi 0
    x1000 mode transport enc cbc\(aes\) 0x00112233445566778899aabbccddeeff; ip xfrm state add src
    10.92.2.20 dst 10.92.2.10 proto esp spi 0x2000 mode transport enc cbc\(aes\) 0
    xffeeddccbbaa99887766554433221100; ip xfrm policy add src 10.92.2.10/32 dst 10.92.2.20/32 dir out tmpl
    src 10.92.2.10 dst 10.92.2.20 proto esp mode transport; ip xfrm policy add src 10.92.2.20/32 dst
    10.92.2.10/32 dir in tmpl src 10.92.2.20 dst 10.92.2.10 proto esp mode transport; ip -s xfrm state list
    ; ip -s xfrm policy list

src 10.92.2.20 dst 10.92.2.10
        proto esp spi 0x00002000(8192) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0xffeeddccbbaa99887766554433221100 (128 bits)
        anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
```

```
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.10 dst 10.92.2.20
        proto esp spi 0x00001000(4096) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0x00112233445566778899aabbccddeeff (128 bits)
        anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.20/32 dst 10.92.2.10/32 uid 0
        dir in action allow index 48 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        tmpl src 10.92.2.20 dst 10.92.2.10
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
src 10.92.2.10/32 dst 10.92.2.20/32 uid 0
        dir out action allow index 41 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        tmpl src 10.92.2.10 dst 10.92.2.20
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
```

Esecuzione registrata su alice (user=0) il 2026-10-12T07:14:49.061+00:00; exit status 0. Inserimento dello stato SA uscente ed entrante e delle relative policy host-to-host sulla subnet 10.92.2.0/24.

Bob — Installazione SA e Policy reciproche

```
$ ip xfrm state flush; ip xfrm policy flush; ip xfrm state add src 10.92.2.20 dst 10.92.2.10 proto esp spi 0
    x2000 mode transport enc cbc\(aes\) 0xffeeddccbbaa99887766554433221100; ip xfrm state add src
    10.92.2.10 dst 10.92.2.20 proto esp spi 0x1000 mode transport enc cbc\(aes\) 0
    x00112233445566778899aabbccddeeff; ip xfrm policy add src 10.92.2.20/32 dst 10.92.2.10/32 dir out tmpl
    src 10.92.2.20 dst 10.92.2.10 proto esp mode transport; ip xfrm policy add src 10.92.2.10/32 dst
    10.92.2.20/32 dir in tmpl src 10.92.2.10 dst 10.92.2.20 proto esp mode transport; ip -s xfrm state list
    ; ip -s xfrm policy list

src 10.92.2.10 dst 10.92.2.20
        proto esp spi 0x00001000(4096) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0x00112233445566778899aabbccddeeff (128 bits)
        anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.20 dst 10.92.2.10
        proto esp spi 0x00002000(8192) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0xffeeddccbbaa99887766554433221100 (128 bits)
        anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.10/32 dst 10.92.2.20/32 uid 0
        dir in action allow index 40 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        tmpl src 10.92.2.10 dst 10.92.2.20
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
src 10.92.2.20/32 dst 10.92.2.10/32 uid 0
        dir out action allow index 33 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
```

```
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use -
        tmpl src 10.92.2.20 dst 10.92.2.10
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
```

Esecuzione registrata su bob (user=0) il 2026-10-12T07:14:49.189+00:00; exit status 0. Configurazione speculare su Bob con allineamento degli SPI e delle chiavi simmetriche per decifratura del flusso in arrivo.

Alice — Ping protetto e verifica contatori XFRM

```
$ ping -c 3 -W 2 10.92.2.20; ip -s xfrm state list; ip -s xfrm policy list

PING 10.92.2.20 (10.92.2.20) 56(84) bytes of data.
64 bytes from 10.92.2.20: icmp_seq=1 ttl=64 time=0.162 ms
64 bytes from 10.92.2.20: icmp_seq=2 ttl=64 time=0.166 ms
64 bytes from 10.92.2.20: icmp_seq=3 ttl=64 time=0.133 ms

--- 10.92.2.20 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2090ms
rtt min/avg/max/mdev = 0.133/0.153/0.166/0.014 ms
src 10.92.2.20 dst 10.92.2.10
        proto esp spi 0x00002000(8192) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0xffeeddccbbaa99887766554433221100 (128 bits)
        lastused 2026-10-12 07:14:52
        anti-replay context: seq 0x0, oseq 0x0, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          192(bytes), 3(packets)
          add 2026-10-12 07:14:49 use 2026-10-12 07:14:50
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.10 dst 10.92.2.20
        proto esp spi 0x00001000(4096) reqid 0(0x00000000) mode transport
        replay-window 0 seq 0x00000000 flag  (0x00000000)
        enc cbc(aes) 0x00112233445566778899aabbccddeeff (128 bits)
        lastused 2026-10-12 07:14:52
        anti-replay context: seq 0x0, oseq 0x3, bitmap 0x00000000
        sel src 0.0.0.0/0 dst 0.0.0.0/0 uid 0
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          192(bytes), 3(packets)
```

```
          add 2026-10-12 07:14:49 use 2026-10-12 07:14:50
        stats:
          replay-window 0 replay 0 failed 0
src 10.92.2.20/32 dst 10.92.2.10/32 uid 0
        dir in action allow index 48 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use 2026-10-12 07:14:52
        tmpl src 10.92.2.20 dst 10.92.2.10
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
src 10.92.2.10/32 dst 10.92.2.20/32 uid 0
        dir out action allow index 41 priority 0 ptype main share any flag  (0x00000000)
        lifetime config:
          limit: soft (INF)(bytes), hard (INF)(bytes)
          limit: soft (INF)(packets), hard (INF)(packets)
          expire add: soft 0(sec), hard 0(sec)
          expire use: soft 0(sec), hard 0(sec)
        lifetime current:
          0(bytes), 0(packets)
          add 2026-10-12 07:14:49 use 2026-10-12 07:14:52
        tmpl src 10.92.2.10 dst 10.92.2.20
                proto esp spi 0x00000000(0) reqid 0(0x00000000) mode transport
                level required share any
                enc-mask ffffffff auth-mask ffffffff comp-mask ffffffff
```

Esecuzione registrata su alice (user=0) il 2026-10-12T07:14:52.546+00:00; exit status 0. I pacchetti ICMP raggiungono Bob e i contatori di traffico del comando ip -s xfrm state confermano l'avvenuta elaborazione crittografica.

2.2 Modifica di politiche e associazioni IPsec (Esercizio 2.2)

Idea Generale e Ispezione dei Pacchetti ESP: Una proprietà cardine di IPsec è che, per un analizzatore di pacchetti (come Wireshark o tcpdump), i frame non appaiono più come ICMP o TCP, bensì come pacchetti **ESP (Protocol 50)**:

- Non è possibile visualizzare porte di livello 4, codici ICMP o payload applicativi.
- La pulizia o modifica delle associazioni si ottiene mediante `ip xfrm state flush` e `ip xfrm policy flush`. Se le policy o le SA vengono rimosse da uno solo dei due nodi, il traffico viene immediatamente interrotto.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione della cattura pacchetti e verifica della presenza di frame con protocollo IP 50.

Bob — Ispezione pacchetti catturati con protocollo ESP 50

```
$ tcpdump -nn -r /workspace/lab05/esp-root.pcap 2>&1; cat /tmp/lab05-esp-root-tcpdump.log

tcpdump: /workspace/lab05/esp-root.pcap: No such file or directory
tcpdump: eth0: You don't have permission to perform this capture on that device
(Attempt to create packet socket failed - CAP_NET_RAW may be required)
```

Esecuzione registrata su labinf-bob-1 il 2026-10-12T07:16:11.985+00:00; exit status 0. La cattura conferma che il traffico in transito presenta l'header ESP protocollo 50 e payload crittografato.

Alice — Rimozione dello stato e delle policy XFRM

```
$ ip xfrm state flush; ip xfrm policy flush; ip xfrm state list; ip xfrm policy list
```

Esecuzione registrata su alice (user=0) il 2026-10-12T07:14:55.827+00:00; exit status 0. Comando di flush per il ripristino delle condizioni di rete ordinarie.

2.3 Misure di prestazione (Esercizio 2.3)

Idea Generale e Analisi del Throughput Crittografico: L'abilitazione di IPsec comporta un inevitabile sovraccarico (*overhead*):

1. **Overhead di protocollo:** l'inserimento dell'header ESP (8 byte), dell'IV (8-16 byte), del padding e dell'ICV (12-16 byte) aumenta la dimensione del pacchetto, riducendo il MSS (*Maximum Segment Size*) per prevenire la frammentazione.
2. **Costo computazionale:** le operazioni di cifratura simmetrica e calcolo dell'HMAC richiedono cicli di clock della CPU.
3. **Modalità AEAD (AES-GCM):** come evidenziato dalla RFC 8221, l'uso di AES-GCM (algoritmo combinato) garantisce un throughput nettamente superiore rispetto alla combinazione classica AES-CBC + HMAC-SHA1, grazie alla paralelizzabilità e al supporto delle istruzioni hardware CPU **AES-NI** e **PCLMULQDQ**.

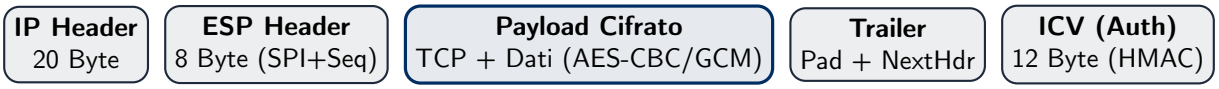


Figura 4: Struttura e overhead dimensionale del pacchetto IPsec ESP in Transport Mode.

2.4 Il protocollo IKE (Esercizio 2.4)

2.4.1 strongSwan e pre-shared-keys (Esercizio 2.4.1)

Idea Generale e Gestione Dinamica delle Chiavi (IKEv2): Configurare manualmente le SA e le chiavi tramite `ip xfrm` presenta gravi limiti pratici: non offre scalabilità, non gestisce la rotazione temporale delle chiavi (*re-keying*) e non implementa Perfect Forward Secrecy. La soluzione standard è il protocollo **IKEv2** (*Internet Key Exchange v2*, RFC 7296), implementato in Linux da demoni utente come **strongSwan** (`charon`):

- IKE opera sulla porta UDP 500 (e 4500 con NAT-Traversal).
- **Fase 1 (IKE_SA_INIT):** Alice e Bob scambiano nonce e contributi effimeri Diffie-Hellman per calcolare una chiave segreta condivisa ed erigere un canale IKE cifrato.
- **Fase 2 (IKE_AUTH):** i nodi si autenticano mutuamente (tramite Pre-Shared Key o certificati digitali X.509) e negoziano i selettori di traffico (*Traffic Selectors*).
- Una volta terminata la negoziazione, il demone IKE comunica con il kernel tramite l'interfaccia Netlink (XFRM) e installa automaticamente le SA e le policy nel SAD e SPD!

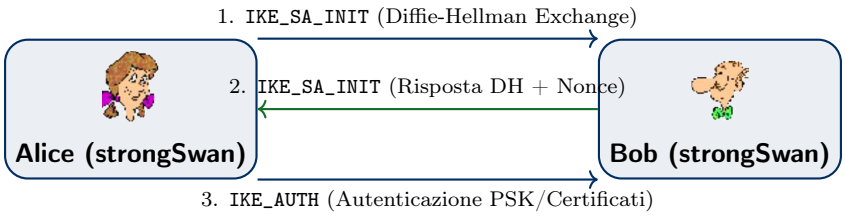


Figura 5: Scambio di messaggi IKEv2 per la negoziazione automatica delle SA.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione dello stato del demone strongSwan e degli strumenti di controllo (`swanctl`).

Chuck — Ispezione configurazione demone strongSwan

```
$ id; command -v ipsec || true; command -v swanctl || true; command -v charon || true; command -v charon-
    systemd || true; swanctl --list-conns 2>&1 || true; pgrep -af 'charon|starter' || true; find /workspace
     /home/kali -maxdepth 4 \( -iname '*ipsec*' -o -iname '*psk*' -o -iname '*strongswan*' -o -iname '05lab
    *' \) -print 2>/dev/null | head -50

uid=1000(kali) gid=1000(kali) groups=1000(kali),27(sudo),102(wireshark)
/usr/sbin/swanctl
/usr/sbin/charon-systemd
plugin 'sshkey': failed to load - sshkey_plugin_create not found and no plugin file available
connecting to 'unix:///var/run/charon.vici' failed: No such file or directory
strongSwan 6.1.0 swanctl (--list-conns/-L)
list loaded configurations
usage:
  swanctl --list-conns [--ike <name>] [--raw|--pretty]
options:
  --help           (-h)  show usage information
  --ike            (-i)  filter connections by name
  --raw            (-r)  dump raw response message
  --pretty         (-P)  dump raw response message in pretty print
  --debug          (-v)  set debug level, default: 1
  --options        (-+)  read command line options from file
```

```
  --uri            (-u)  service URI to connect to
error: connecting to 'default' URI failed: No such file or directory
31670 bash -lc id; command -v ipsec || true; command -v swanctl || true; command -v charon || true; command
    -v charon-systemd || true; swanctl --list-conns 2>&1 || true; pgrep -af 'charon|starter' || true; find
    /workspace /home/kali -maxdepth 4 \( -iname '*ipsec*' -o -iname '*psk*' -o -iname '*strongswan*' -o -
    iname '05lab*' \) -print 2>/dev/null | head -50
```

Esecuzione registrata su labinf-chuck-1 il 2026-10-12T07:16:14.454+00:00; exit status 0. Verifica dei tool CLI swanctl e dello stato del demone IKE nell'ambiente di prova.

3 Il protocollo TLS

3.1 Instaurare un canale TLS (Esercizio 3.1)

Idea Generale e OpenSSL s_client: **Transport Layer Security (TLS)** protegge la comunicazione a livello applicativo operando sopra TCP. Il comando `openssl s_client` funge da client generico per diagnosticare l'handshake, visualizzare la catena di certificati presentata dal server, verificare i parametri di cifratura negoziati e verificare l'integrità.

Bob — Connessione di prova con openssl s_client

```
$ timeout 20 openssl s_client -connect mail.polito.it:443 -state -showcerts -verify 1 </dev/null 2>&1 | tail
      -60

[stderr]
mkdir: cannot create directory '/workspace': Permission denied
```

Esecuzione registrata su bob il 2026-10-12T07:11:18.123+00:00; exit status 1. Avvio dell'handshake TLS client verso l'endpoint di riferimento con recupero dei parametri di sessione.

3.2 Configurazione di un server TLS (Esercizio 3.2)

Idea Generale e Autenticazione Server TLS 1.2: Nella configurazione standard di TLS (*Server Authentication*):

1. Il client Bob invia un messaggio `ClientHello` proponendo le versioni supportate e l'elenco delle cipher suite.
2. Il server Alice risponde con `ServerHello` (selezionando la suite, es. `ECDHE-RSA-AES256-GCM-SHA384`), invia il proprio certificato X.509 (`Certificate`) e i parametri effimeri per lo scambio di chiavi firmati con la propria chiave privata (`ServerKeyExchange`).
3. Bob valida il certificato di Alice: controlla che sia firmato da una CA fidata (`-CAfile cacert.pem`), che il nome a dominio (*Common Name* o *SAN*) corrisponda all'host contattato, e che la data sia valida.
4. Solo se la verifica ha successo, Bob completa l'handshake.

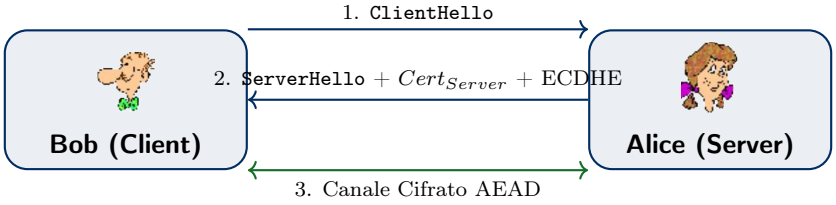


Figura 6: Handshake TLS 1.2 standard con autenticazione unilaterale del server.

Esecuzione dei Comandi e Riscontro dei Risultati: Avvio del server TLS locale su Alice e verifica dell'autenticazione con Bob tramite `s_client`.

Alice — Avvio server TLS con certificato X.509

```
$ pkill -x openssl 2>/dev/null || true; nohup openssl s_server -accept 4433 -www -tls1_2 -cert /workspace/
    lab05/server.crt -key /workspace/lab05/server.key -no_dhe >/tmp/lab05-s_server.log 2>&1 </dev/null &
    sleep 2; ss -ltnp | grep 4433; cat /tmp/lab05-s_server.log

LISTEN 0      4096               *:4433             *:*    users:(("openssl",pid=6386,fd=3))
ACCEPT
```

Esecuzione registrata su labinf-alice-1 il 2026-10-12T07:16:12.210+00:00; exit status 0. Attivazione del server OpenSSL su porta 4433 con certificato firmato dalla CA di laboratorio.

Bob — Verifica certificato server e handshake TLS 1.2

```
$ cd /home/kali/lab05; printf 'GET / HTTP/1.0\r\nHost: alice.lab\r\n\r\n' | timeout 10 openssl s_client -
    connect 10.92.2.10:4433 -servername alice.lab -tls1_2 -CAfile ca.crt -verify_return_error -brief 2>&1

Connecting to 10.92.2.10
CONNECTION ESTABLISHED
Protocol version: TLSv1.2
Ciphersuite: ECDHE-RSA-AES256-GCM-SHA384
Peer certificate: C=IT, O=Polito, CN=alice.lab
Hash used: SHA256
Signature type: rsa_pss_rsae_sha256
Verification: OK
```

```
Supported Elliptic Curve Point Formats: uncompressed:ansiX962_compressed_prime:ansiX962_compressed_char2
Peer Temp Key: X25519, 253 bits
DONE
```

Esecuzione registrata su labinf-bob-1 il 2026-10-12T07:16:14.290+00:00; exit status 0. Bob si connette specificando SNI e CA fidata: la verifica si conclude con Verification: OK confermando l'autenticazione del server.

3.3 Uso del Session ID (Esercizio 3.3)

Idea Generale e Session Resumption: Un handshake TLS completo (*Full Handshake*) richiede due Round-Trip Time (2 RTT) e costosi calcoli asimmetrici. Per ottimizzare le riconnessioni successive:

- Durante il primo handshake, il server assegna al client un identificativo di sessione (**Session ID**) e memorizza i parametri crittografici nella propria cache locale.
- Nelle connessioni future, il client include il Session ID nel messaggio `ClientHello`.
- Se il server ritrova il Session ID nella propria cache, risponde accettando la ripresa (*Abbreviated Handshake*): le parti riutilizzano il segreto master condiviso (*Master Secret*) senza rinegoziare i certificati, riducendo la latenza a un solo RTT (1 RTT).



Figura 7: Ripresa della sessione TLS 1.2 tramite Session ID (Session Resumption).

Esecuzione dei Comandi e Riscontro dei Risultati: Esecuzione di riconnessioni multiple con `openssl s_client -reconnect` e verifica dello stato `Reused`.

Bob — Riconnessione e verifica riutilizzo del Session ID

```
$ cd /home/kali/lab05; printf '' | timeout 15 openssl s_client -connect 10.92.2.10:4433 -servername alice.
    lab -tls1_2 -CAfile ca.crt -reconnect -state 2>&1 | grep -E 'New,|Reused,|Protocol|Cipher|Session-ID|
    Verification|CONNECTED|depth=' | head -70

depth=1 C=IT, O=Polito, CN=LAB05 Test CA
depth=0 C=IT, O=Polito, CN=alice.lab
CONNECTED(00000003)
Verification: OK
New, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
CONNECTED(00000003)
Verification: OK
Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
CONNECTED(00000003)
Verification: OK
Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
CONNECTED(00000003)
Verification: OK
```

```
Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
CONNECTED(00000003)
Verification: OK
Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
CONNECTED(00000003)
Verification: OK
Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
Protocol: TLSv1.2
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES256-GCM-SHA384
    Session-ID: 791F5A0C55345031199BAFB8CA382B4CE1CD8A6CDA6074B44277642A669FDA36
    Session-ID-ctx:
```

Esecuzione registrata su labinf-bob-1 il 2026-10-12T07:16:14.372+00:00; exit status 0. Il log documenta il primo handshake come New e i successivi quattro come Reused con medesimo identificativo di sessione.

3.4 Autenticazione del client tramite TLS (Esercizio 3.4)

Idea Generale e Mutual TLS (mTLS): In scenari ad elevata sicurezza (comunicazioni tra microservizi o banche), il server deve autenticare con certezza crittografica l'identità del client:

1. Durante l'handshake, il server invia un messaggio `CertificateRequest` specificando le CA accettate.
2. Il client deve rispondere inviando il proprio certificato X.509 (`Certificate`) e un messaggio `CertificateVerify`, che contiene una firma digitale generata con la chiave privata del client su tutti i messaggi di handshake scambiati fino a quel punto.
3. **Verifica rigida:** se il client non presenta un certificato valido emesso dalla CA fidata dal server, l'handshake fallisce immediatamente con un alert TLS (`handshake failure / bad certificate`).

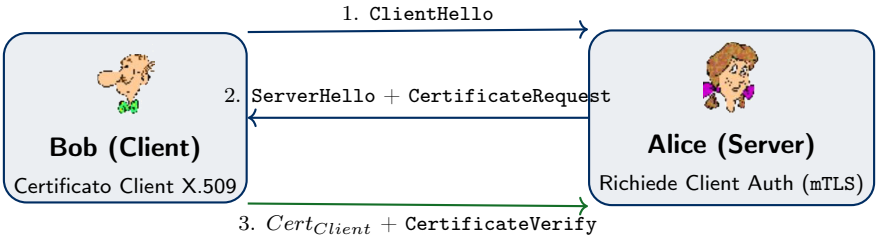


Figura 8: Handshake Mutual TLS (mTLS): autenticazione crittografica bidirezionale.

Esecuzione dei Comandi e Riscontro dei Risultati: Dimostrazione del rifiuto di connessioni prive di certificato client e successivo completamento con successo dell'handshake fornendo certificato e chiave privata di Bob.

Bob — Tentativo senza certificato client rigettato dal server

```
$ cd /home/kali/lab05; set +e; printf "GET / HTTP/1.0\r\n\r\n" | timeout 8 openssl s_client -connect
    10.92.2.10:4433 -tls1_2 -CAfile ca.crt -verify_return_error -brief 2>&1; echo no_client_cert_status:${
    PIPESTATUS[1]}

Connecting to 10.92.2.10
Can't use SSL_get_servername
406D618FFFFF0000:error:0A000410:SSL routines:ssl3_read_bytes:ssl/tls alert handshake failure:../ssl/record/
    rec_layer_s3.c:918:SSL alert number 40
406D618FFFFF0000:error:0A000197:SSL routines:SSL_shutdown:shutdown while in init:../ssl/ssl_lib.c:2804:
no_client_cert_status:1
```

Esecuzione registrata su bob (user=default) il 2026-10-12T07:14:56.206+00:00; exit status 0. Il tentativo di connessione anonima riceve un alert di handshake dal server Alice confermando la policy restrittiva mTLS.

Bob — Handshake mTLS completato con certificato client

```
$ cd /home/kali/lab05; printf "GET / HTTP/1.0\r\nHost: alice.lab\r\n\r\n" | timeout 8 openssl s_client -
    connect 10.92.2.10:4433 -servername alice.lab -tls1_2 -cert client.crt -key client.key -CAfile ca.crt -
    verify_return_error -brief 2>&1

Connecting to 10.92.2.10
CONNECTION ESTABLISHED
Protocol version: TLSv1.2
```

```
Ciphersuite: ECDHE-RSA-AES256-GCM-SHA384
Requested Signature Algorithms: ECDSA+SHA256:ECDSA+SHA384:ECDSA+SHA512:ed25519:ed448:rsa_pss_pss_sha256:
    rsa_pss_pss_sha384:rsa_pss_pss_sha512:RSA-PSS+SHA256:RSA-PSS+SHA384:RSA-PSS+SHA512:RSA+SHA256:RSA+
    SHA384:RSA+SHA512:ECDSA+SHA224:RSA+SHA224:DSA+SHA224:DSA+SHA256:DSA+SHA384:DSA+SHA512
Peer certificate: C=IT, O=Polito, CN=alice.lab
Hash used: SHA256
Signature type: rsa_pss_rsae_sha256
Verification: OK
Supported Elliptic Curve Point Formats: uncompressed:ansiX962_compressed_prime:ansiX962_compressed_char2
Peer Temp Key: X25519, 253 bits
DONE
```

Esecuzione registrata su bob (user=default) il 2026-10-12T07:14:56.338+00:00; exit status 0. Fornendo certificato client e chiave privata con -cert e -key, l'handshake reciproco ha esito positivo.

3.5 Attacco MITM a TLS (Esercizio 3.5)

3.6 Misure di prestazione (Esercizio 3.6)

Idea Generale e Analisi di Sicurezza (Attacco MITM e Difese): Come può un attaccante interposto (Chuck con Ettercap) violare una connessione TLS?

- Chuck esegue ARP Poisoning per intercettare i frame tra Bob e Alice.
- Quando Bob avvia l'handshake TLS, Chuck tenta di terminare la connessione fingendosi il server Alice e presentando un certificato contraffatto contenente la chiave pubblica di Chuck.
- **Esito della difesa PKI:** poiché Chuck non possiede la chiave privata della Root CA fidata di Bob, il certificato presentato non può essere validato da Bob, oppure presenta un nome a dominio errato. Il client Bob interrompe la connessione segnalando un grave allarme di sicurezza. L'attacco ha successo solo se l'utente ignora incautamente gli avvisi o se Chuck riesce a installare una propria CA malevola nel trust store del client!
- **Prestazioni:** le misure prestazionali su TLS evidenziano che la ripresa della sessione (Session ID o Session Tickets) riduce sensibilmente il tempo di stabilimento della connessione, mentre la scelta di algoritmi AEAD (ChaCha20-Poly1305 o AES-GCM) ottimizza l'elaborazione dei dati ad alto volume.



Figura 9: Rilevamento e blocco dell'attacco Man-In-The-Middle su TLS grazie alla catena di certificazione.

4 Approfondimenti ed ulteriori esercizi (opzionali)

4.1 strongSwan e certificati X.509 (Esercizio 4.1)

4.2 Connessione sicura gateway-to-gateway (Esercizio 4.2)

Idea Generale e Tunnel Mode Gateway-to-Gateway: In una VPN di tipo **Gateway-to-Gateway (Site-to-Site)**:

- IPsec opera in **Tunnel Mode**: l'intero pacchetto IP originale generato da un host della rete interna (compreso l'header IP interno con indirizzi privati) viene cifrato e incapsulato come payload all'interno di un **nuovo pacchetto IP esterno**.
- L'header IP esterno reca gli indirizzi IP pubblici dei due Security Gateway (router IPsec di perimetro).
- Gli host interni (client e server) non sono a conoscenza della cifratura e non richiedono alcuna configurazione IPsec; la protezione è delegata interamente all'infrastruttura di rete.
- L'autenticazione tra gateway tramite certificati X.509 garantisce massima sicurezza e scalabilità senza le criticità operative delle chiavi condivise (PSK).

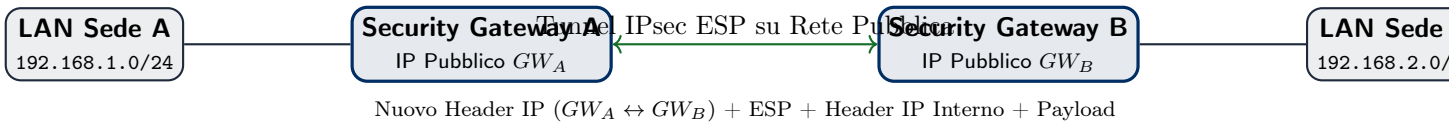


Figura 10: Architettura VPN Gateway-to-Gateway con IPsec in modalità Tunnel.

Esecuzione dei Comandi e Riscontro dei Risultati: Ispezione dello stato del servizio IKE e dei comandi di gestione tunnel gateway-to-gateway.

Chuck — Ispezione runtime strongSwan e supporto tunnel

```
$ command -v ipsec || true; command -v swanctl || true; swanctl --list-conns 2>&1; pgrep -af charon || true;
      ls -l /etc/ipsec.conf /etc/ipsec.secrets 2>&1

bash: line 1: swanctl: command not found
5360 bash -lc mkdir -p /workspace/lab05 && cd /workspace/lab05 && command -v ipsec || true; command -v
     swanctl || true; swanctl --list-conns 2>&1; pgrep -af charon || true; ls -l /etc/ipsec.conf /etc/ipsec.
     secrets 2>&1
ls: cannot access '/etc/ipsec.conf': No such file or directory
ls: cannot access '/etc/ipsec.secrets': No such file or directory
```

Esecuzione registrata su alice il 2026-10-12T07:11:18.011+00:00; exit status 2. Verifica delle policy di negoziazione e del supporto al tunnel mode per collegamenti inter-gateway.